Ferramentas

para a construção de compiladores



Ciência da Computação

Ferramentas

- Inúmeras ferramentas:
  - https://en.wikipedia.org/wiki/Comparison_of_parser_generators

- Nesta aula:
  - Flex
  - Bison

Flex

- O Flex (*Fast Lexical Analyzer Generator*) é uma ferramenta gratuita e open-source para geração de analisadores léxicos.

- Instalação (Debian/Ubuntu):

```
sudo apt-get update
sudo apt-get install flex
```

- Uso:

```
flex lexico.l
```

Flex

- O arquivo é dividido em três partes, separadas por linhas que começam com %%:

```
[Definições]

%%

[Regras]

%%

[Código do usuário]
```

Flex

```
definições

%%

regras

%%

código do usuário
```

Definições (2 partes):
1- Delimitada pelos símbolos “%{” e “%}”, são colocadas as instruções da linguagem C que posteriormente serão incluídas
no início do arquivo C gerado pelo FLEX.

2- Expressões Regulares - macros para as expressões regulares mais comuns como por exemplo algarismo ou letra do alfabeto.
Exemplo:

```
DIGITO [0-9]
IDENTIFICADOR [a-z][a-z0-9]*
```

Flex

```
definições

%%

regras

%%

código do usuário
```

Regras:
A seção de Regras contém um conjunto de regras que definem os tipos léxicos que serão entendidos pelo analisador.

O Flex usa essas regras para gerar o código do analisador.

Formato:
`[padrão] [ação]`
um novo padrão tem que aparecer sempre em uma nova linha, sem indentação, e a definição da ação correspondente tem que começar na mesma linha

Flex

```
definições

%%

regras

%%

código do usuário
```

Código do usuário:
Na seção de Código do Usuário, você pode colocar a função `main()` do seu programa ou outras funções que você desejar

Todo o código que é colocado nessa seção, juntamente com o que é colocado entre %{ %}, é copiado para o arquivo de saída gerado pelo Flex

Flex - Exemplo

```
/* Conta linhas, palavras e caracteres de um arquivo texto
cujo o nome sera passado via linha de comando */
%{
int contCaracter = 0, contPalavra = 0, contLinha = 0;
%}
EOL \n
PALAVRA [^ \n\t]+
%%
{PALAVRA} {contPalavra++; contCaracter += yyleng;}
{EOL} {contCaracter++; contLinha++;}
. {contCaracter++;}
%%
int main(int argc,char** argv){
yyin = fopen(argv[1],"r"); /* abre arq de entrada */
yylex(); /* executa o scanner */
printf("# linhas: %d, # palavras: %d, # caracteres: %d\n",
contLinha, contPalavra, contCaracter);
fclose(yyin); /* fecha arq de entrada */
}
```

yylex() evoca o analisador léxico gerado pelo flex que processará as expressões regulares

char ***yytext** : é um apontador para uma cadeia de caracteres contendo o último item léxico reconhecido

int **yyleng**: é o comprimento do último item léxico reconhecido

FILE ***yyin**: é o descritor do arquivo de entrada que será processado

Flex - Exemplos

Exemplo1: pascal.l

1) Gerar o arquivo C:

```
flex pascal.l
```

2) Compilar código C:

```
gcc lex.yy.c -o lexico_pascal -lfl
```

3) Executar:

```
./lexico_pascal
<entrar com sequencia de tokens para avaliação>
```

Flex - Exemplos

Exemplo2: c.l

1) Gerar o arquivo C:

```
flex c.l
```

2) Compilar código C:

```
gcc lex.yy.c -o lexico_c -lfl
```

3) Executar:

```
./lexico_pascal < c.in
```

Flex - Referências

https://blog.xrds.acm.org/2017/12/flex-lexical-analysis/

https://people.iith.ac.in/ramakrishna/Compilers-Aug14/doc/flex.pdf

http://alumni.cs.ucr.edu/~lgao/teaching/flex.html

https://www.di.uminho.pt/~prh/FlexTutorial.pdf

Bison/Yacc

- O UNIX original tinha como um de seus componentes, o YACC – Yet Another Compiler Compiler
- O YACC gera um analisador sintático (= parser) a partir de uma especificação formal de uma gramática
- O GNU Bison é a implementação da FSF (Free Software Foundation) para o YACC
- Usando o Bison, podemos criar de simples programas interativos (como calculadoras) a analisadores de linguagens de programação bem complexas

Bison/Yacc

- Instalação (Debian/Ubuntu):

```
sudo apt-get update
sudo apt-get install bison
```

- Uso:

```
yacc parser.y
```

Bison/Yacc

- Estrutura do arquivo

```
%{
Prólogo
%}
Declarações do Bison
%%
Regras gramaticais
%%
Epílogo
```

Prólogo – contém definições iniciais em C que são jogadas diretamente para o arquivo de saída do Bison, antes da definição da função yyparse

Declarações do Bison – onde são definidos símbolos terminais e não-terminais e relações de precedência. Em gramáticas simples, pode ser deixado vazio

Regras gramaticais – é o coração do arquivo. É obrigatório ter pelo menos 1 regra gramatical

Epílogo – é copiado *ipsis literis* para o arquivo de saída

Bison - Exemplos

Ver exemplos calc.l e calc.y – Calculadora

1) **`make`**
`(Makefile incluso)`

2) **`./calc`**
`3+4`

YACC/Bison - Referências

- https://www.inf.ufrgs.br/~johann/comp/lexyacc.pdf

- http://web.mit.edu/gnu/doc/html/bison_8.html